



Département mathématiques et informatique
UFR des Sciences

TP4

“Classes de types et fonctions d’ordre supérieur”

Exercice 1 : Classes types

En considérant une représentation d’un ensemble par une liste de type quelconque. Écrire les fonctions suivantes en donnant la possibilité d’instancier l’ensemble avec un type différent :

1. appartenance d’un élément dans un ensemble,
2. supprimer les doublons dans un ensemble en utilisant appartient
3. supprimer dans un ensemble les éléments inférieurs à un nombre donné.
4. inclusion de deux ensembles,
5. intersection de deux ensembles,
6. union de deux ensembles (faire la différence avant),
7. somme sur les éléments dans l’intersection s’il s’agit des nombres naturels, entiers ou réels.

Noter qu’aucun élément ne figure plus d’une fois. l’ensemble vide est représenté par la liste vide.

Exercice 2 : Curryfication

Nous considérons une application de restaurant qui permet aux clients de personnaliser leurs commandes de pizzas. Chaque pizza peut avoir différentes garnitures, et le prix de la pizza dépend des garnitures sélectionnées. Votre tâche est de définir une fonction qui calcule le prix total d’une pizza personnalisée.

1. Définir une fonction `calculerPrixPizza` qui prend trois paramètres : le prix de base de la pizza, le prix par garniture et le nombre de garnitures. La fonction doit retourner le prix total de la pizza.
2. Définir une fonction partiellement appliquée `calculerPrixPizzaGrande` en fixant le prix de base et le prix par garniture pour une pizza de grande taille (p.ex. prix de base 12.99 et prix garniture 1.5). Votre fonction doit utiliser la fonction définie dans 1.
3. De la même manière, définir une fonction partiellement appliquée `calculerPrixPizzaMoyenne` et `calculerPrixPizzaPetite` en fixant le prix de base et le prix par garniture pour une pizza de taille moyenne (prix de base 9.99 et prix garniture 1.25) et de taille petite (prix de base 7.99 et prix garniture 1.0) respectivement.
4. Utilisez les fonctions partiellement appliquées pour calculer le prix total de différentes pizzas personnalisées, p.ex. pizza grande avec 4 garnitures, pizza moyenne avec 2 garnitures, et pizza petite avec une garniture.

Exercice 3 : fonction d’ordre supérieur (map)

Évaluer et expliquer en commentaire les expressions suivantes (rappel : `elem x xs` détermine si `x` est un élément de la liste `xs`) :

1. `map (*2) [1..6]`
2. `map (\x -> x:[]) [1..5]`
3. `map (\x -> [v | v <- x, even v] ) [[1,2,3],[4,5,6],[7,8,9]]`

Exercice 4 : fonction d'ordre supérieur (map)

En utilisant la fonction d'ordre supérieur `map` et d'autres fonctions de bibliothèque, proposer des définitions pour les fonctions suivantes :

1. Une fonction `caracSuivant :: String -> String` qui remplace chaque caractère de la liste par le caractère suivant dans l'ordre alphabétique.
2. Une fonction `doubles :: [Int] -> [Int]` qui renvoie le double de chaque entier d'une liste.
3. Une fonction `f2cList :: [Float] -> [Float]` qui convertit des degrés Fahrenheit en Celsius : $C = (F - 32)/1,8$. Proposer une version avec le `where` et une autre avec une fonction anonyme.
4. Une version avec ZF-expressions de `caracSuivant` et comparer les résultats avec 1.

Exercice 5 : fonction d'ordre supérieur (map et filter)

Redéfinir toutes les fonctions de l'exercice 1 avec les fonctions d'ordre supérieur (`map`, `filter` et `foldr`) quand c'est possible. Si vous réussissez proposer des définitions avec les ZF-expressions.